IA - Unidad 4

Representación del conocimiento

Hay que hacer una distinción entre los HECHOS (verdades que queremos representar) y las representación DE LOS HECHOS (es lo que se manipula en los programas de IA).

Podemos clasificar estas entidades en dos niveles:

- Nivel del conocimiento, donde se describen los hechos.
- Nivel simbólico, donde se describen los objetos del nivel del conocimiento en términos de símbolos manipulables.

La relación entre los hechos y su representación se conoce como correspondencia de la representación. Es importante el uso de una buena representación, porque esto hace que el mecanismo de razonamiento de un programa sea útil.

Propiedades de un buen sistema de representación de conocimiento

- Suficiencia de la representación: Capaz de representar todos los tipos de conocimiento necesarios.
- Suficiencia deductiva: Puede manipular las estructuras para obtener nuevas estructuras que se correspondan con nuevo conocimiento deducido.
- Eficiencia deductiva: Puede incorporar información adicional en las estructuras de conocimiento.
- Eficiencia en la adquisición: Puede adquirir nueva info. Fácilmente.

Todavía no existe un sistema que optimice todos estos aspectos y que funcione para todo tipo de conocimiento.

Modelos de representación del conocimiento

Tenemos sistemas puramente sintácticos, donde no importa el significado del conocimiento que se representa, con reglas simples para manipular la representación. Por otro lado, hay sistemas puramente semánticos en los que no hay una forma unificada de hacerlo.

1. Conocimiento relacional simple

Los hechos declarativos se pueden representar mediante un conjunto de relaciones, como en una base de datos. Se dice simple porque ofrece poca capacidad deductiva. Por ejemplo tenemos estos hechos:

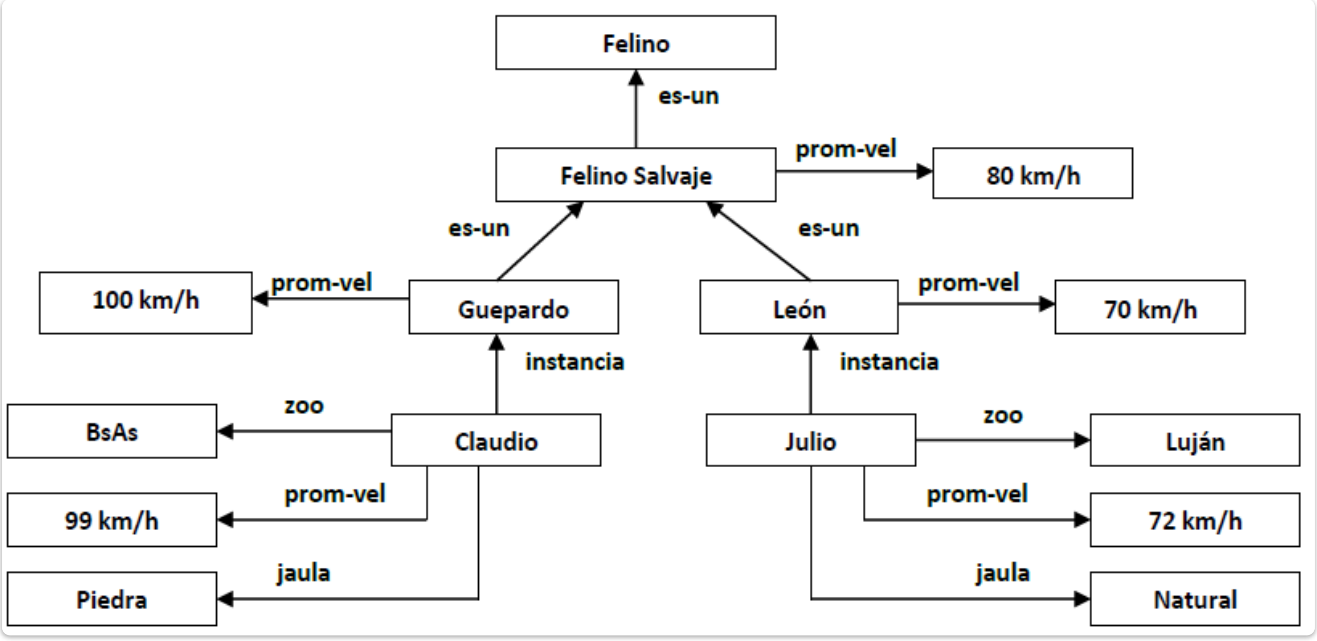
Nombre	Felino Salvaje	Promedio velocidad	Zoo
Julio	León	72	Luján
Claudio	Guepardo	99	BsAs
Tulio	Tigre	80	Luján

Con esto se puede hacer un procedimiento que por ejemplo, diga cuál es el felino más rápido.

2. Conocimiento heredable

Esta representación básica se puede extender con mecanismos de inferencia que operan sobre la estructura de la representación. Una de las formas más útiles de inferencia es la herencia de propiedades, donde los elementos de una clase heredan atributos y valores de otras clases más generales.

Para que funcione, los objetos se deben organizar en clases y las clases en una jerarquía.



- Las líneas representan atributos
- Los nodos representan objetos y valores de sus atributos
- Las flechas conectan objetos con sus valores

Hay dos atributos importantes en general: es-un, que indica que una clase está contenida en otra, y la instancia, que indica pertenencia a una clase. Estos son la base de la herencia de propiedades.

3. Conocimiento deductivo

Es una forma de inferencia que usa la lógica tradicional. Esto sería un ejemplo de la lógica de predicados de primer orden:

$$\forall x: \text{felino_salvaje}(x) \wedge \text{hambriento}(x) \wedge \text{prom_vel}(x,72) \wedge \text{presa}(y) \wedge \text{prom_vel}(y,20) \rightarrow \text{come}(x,y)$$

Igualmente, se necesita un mecanismo de inferencia que pueda aprovechar el conocimiento. Tendría que ser uno que implemente las reglas lógicas de la inferencia. Hay varios, algunos razonan hacia adelante a partir de los hechos hasta las conclusiones, otros razonan hacia atrás.

4. Conocimiento procedimental

Hay un tipo de conocimiento llamado procedimental, que especifica que se hace cuando ocurre determinada situación. Se puede representar de distintas formas, normalmente como un código que hace algo. La máquina utiliza el conocimiento cuando ejecuta el código para realizar alguna tarea determinada.

```
come (X,Y) :- felino_salvaje (X), hambriento (X), prom_vel(X,72),
              presa (Y), prom_vel (Y,20)
```

Características de la representación del conocimiento

1. Atributos importantes

Atributos genéricos que aparecen en distintos dominios de aplicación deben ser tratados adecuadamente. Los dos atributos importantes son instancia y es-un, vistos anteriormente.

2. Relaciones entre atributos

Los atributos que se usan para describir objetos a su vez pueden representarse como entidades. Independientemente del conocimiento que representan, hay 4 propiedades:

- Inversos: Cuando se describen las relaciones como atributos, se establece un punto de vista. Por ejemplo: `zoo(Julio, Lujan)`. Alternativamente, se pueden utilizar atributos que fijan un punto de vista, pero se utilizan por parejas, de manera que uno sea inverso del otros. Entonces para este ejemplo existen dos hechos, uno que se asocia con Julio y otro con Lujan. (???)
- Existencia de una jerarquía es-un: Por ejemplo, hablamos de felinos salvajes, que es una especialización de felinos, que es especialización de mamíferos, y tal y tal.

- Técnicas para el razonamiento acerca de los valores: Los valores de atributos se pueden especificar durante la creación de una base de conocimiento. Pero hay veces donde el sistema tiene que razonar sobre valores que no se recibieron explícitamente. Hay varios tipos de información involucrados, incluyendo información acerca del tipo del valor, restricciones sobre ese valor, reglas para el cómputo de un valor cuando es necesario (reglas hacia atrás), y reglas que describen acciones a realizar si se conoce un valor (reglas hacia adelante).
- Atributos univaluados: Por ejemplo, cierto felino solo pertenece a un zoológico. Si estos atributos cambian, es porque hubo un cambio en el mundo o existe alguna contradicción en la base de conocimiento.

3. Granularidad en la representación

No es fácil elegir la granularidad de la representación para un problema. Mientras más low-level, más fácil razonar en algunos casos, pero sube la complejidad y se necesita más espacio.

4. Representación de conjuntos de objetos

Existen propiedades que se verifican en conjuntos de objetos pero no en elementos individuales. También, si hay una propiedad que verifican todos los elementos de un conjunto, es más eficiente asociar esa propiedad al conjunto en vez de a cada individuo.

Hay dos formas para definir un conjunto y sus elementos:

- Por extensión: Enumero todos los elementos.
- Por comprensión: Creo cierta regla, que devuelve True/False si el objeto pertenece al conjunto o no.

Es fácil definir por extensión, pero si se hace por comprensión se pueden definir conjuntos infinitos o aquellos donde no se conocen todos los elementos.

5. Búsqueda de la estructura adecuada a cada circunstancia

Una vez que se encuentra la estructura de conocimiento adecuada al problema, se debe establecer una correspondencia entre ambos. Los detalles dependen de la representación usada.

Problema del Marco

Para ilustrarlo se puede considerar a un robot doméstico. Se necesitan representar un gran número de objetos y relaciones. Se puede almacenar la descripción de cada estado como una lista de hechos. Pero, puede pasar que las descripciones sean muy largas. La gran mayoría de los hechos no cambian de un estado a otro, aunque se representan en cada nodo. Además, se insumiría mucho tiempo creando dichos nodos y copiando los hechos entre nodos.

Ese problema de tener que representar los hechos que cambian y los que no, se conoce como problema del marco.

En algunos dominios solo hace falta representar todos los hechos, en otros no es fácil determinar cuáles cambian. Por ejemplo si tenemos una planta encima de una mesa debajo de una ventana, si la mesa pasa a estar en el centro de la habitación, tambien se debe inferir que la planta está en el centro de la habitación, pero no la ventana.

Para hacer este razonamiento, se pueden usar axiomas de marco, que describen las cosas que cambian cuando se aplica cierto operador en un estado.

Ejemplo:

```
Color(x,y,s1) ^ desplaza(x,s1,s2) → color(x,y,s2)
```

"Si x es de color y en el estado s1, y se aplica la operación de desplazar x en el estado s1 para llegar al s2, entonces el color de x en s2 sigue siendo y"

Lamentablemente en dominios complejos hace falta usar un número muy grande de estos axiomas. Si no, se puede suponer que solo cambia lo que **debe** hacerlo, lo que debe serían los cambios que provocan los axiomas o se deducen lógicamente.

Lógica

Lógica y lenguaje

Y que se supone que es la lógica entonces

Es el estudio de métodos y principios usados al distinguir argumentos correctos de los incorrectos.

Muchas veces se define como la ciencia del razonamiento, pero no es muy exacta. El razonamiento es una clase especial de pensamiento llamada inferencia, donde se sacan conclusiones a partir de premisas. Y esto no es un campo exclusivo de la lógica.

Naturaleza del argumento

En la inferencia se afirma una proposición sobre la base de otra/s. **Las proposiciones son verdaderas o falsas**.

A cada inferencia corresponde un **argumento**. Un argumento es un grupo de proposiciones o enunciados de los que se afirma que hay uno que se sigue de los demás. **Todo argumento tiene una estructura (premisa y conclusión).**

Toda proposición puede ser premisa o conclusión dependiendo del contexto.

Podemos tambien distinguir entre argumentos deductivos e inductivos. Solo en uno deductivo se pretende que las premisas provean un fundamento 100% concluyente.

Hay que notar que en vez de usar "correcto" o "incorrecto", se usan los términos técnicos "válido" o "inválido".

Un argumento deductivo es válido si sus premisas y conclusiones se relacionan de manera que sea IMPOSIBLE que las premisas sean verdaderas, a menos que la conclusión lo sea tambien.

Verdad y validez

La verdad/falsedad caracterizan las proposiciones. La validez/invalidez caracterizan los argumentos.

Entre estos términos hay una conexión que no es simple. Algunos argumentos válidos solo tienen proposiciones verdaderas. Otros, puede tener proposiciones exclusivamente falsas y ser válido.

Entonces la validez de un argumento no garantiza la verdad de su conclusión.

Y, por otro lado, la falsedad de la conclusión no garantiza la invalidez del argumento, PERO la falsedad de su conclusión garantiza que el argumento es inválido O que por lo menos una premisa es falsa.

En resumen, dos condiciones tiene que satisfacer un argumento para establecer la verdad de su conclusión: Debe ser válido y sus premisas deben ser todas verdaderas.

Lógica simbólica

La lógica, como las matemáticas, tiene un sistema de notación técnica especial. Los símbolos especiales permiten exhibir las estructuras lógicas de los argumentos de una forma más clara. Se adaptan mucho mejor a la obtención de las inferencias.

Argumentos que contienen enunciados compuestos

Enunciados simples y compuestos

Un enunciado simple no contiene otro enunciado como parte componente, un enunciado compuesto si lo hace. Estas partes componentes pueden ser tambien enunciados compuestos.

Algunas maneras de combinar los enunciados en compuestos:

Conjunción: insertando la palabra Y. "Escribo esto Y estoy aburrido". Dados p y q, enunciados cualquiera, su conjunción es `p . q`. La conjunción es verdadera cuando sus conjuntos son ambos verdaderos, de lo contrario es falsa.

Su tabla de verdad:

p	q	p.q
T	T	T
T	F	F
F	T	F
F	F	F

Negación: Dado un enunciado p, su negación es `~p`.

p	~p
T	F
F	T

Disyunción: Combina dos enunciados con la palabra O. Tiene dos sentidos diferentes:

- Débil o inclusivo: sería en realidad un y/o
- Fuerte o exclusivo: Al menos uno de los disyuntivos es verdadero, pero no ambos al mismo tiempo.

La disyunción débil se escribe `p v q`.

p	q	pvq
T	T	T
T	F	T
F	T	T
F	F	F

Una disyunción exclusiva se escribe como `(p v q) . ~(p . q)`.

p	q	$p \veebar q$
V	V	F
V	F	V
F	V	V
F	F	F

Enunciados condicionales

Por ejemplo: *"Si el tren se retrasa entonces perderemos nuestro transbordo"*.

El enunciado entre el sí y él entonces es el antecedente, el que sigue es el consecuente.

`p -> q`

Un condicional no afirma que el antecedente sea verdadero o que el consecuente lo sea, solo que si el antecedente es verdadero, el consecuente tambien lo es. El antecedente implica su consecuente. Cualquier condicional de antecedente verdadero y consecuente falso debe ser falso. Para que el condicional sea verdadero, la condicion indicada debera ser falsa. Pasandolo a tabla de verdad:

p	q	~q	p. ~q	~(p. ~q)	$p \supset q$
T	T	F	F	T	T
T	F	T	T	F	F
F	T	F	F	T	T
F	F	T	F	T	T

Formas de argumentos y tablas de verdad

La validez o no de un argumento son características formales del mismo. Si dos argumentos tienen la misma forma, ambos tienen que ser válidos o inválidos, no importa el contenido.

Dadas distintas variables sentenciales (ej.: p, q, r, s, ...) se puede definir una forma argumental, que es un arreglo de símbolos que tiene variables sentenciales. Así se consigue un argumento.

Un argumento que resulte de una sustitución de enunciados en lugar de variables sentenciales de una forma argumental, es una **instancia de sustitución** de esa forma argumental.

La **forma específica de un argumento** dado es aquella forma argumental de la cual resulta el argumento reemplazando cada variable por un enunciado simple diferente (p, q, ...).

Entonces, si la forma específica de un argumento tiene una instancia de sustitución con premisas verdaderas y conclusión falsa, el argumento es inválido. Esto es la **técnica de refutación por analogía lógica**.

Si se quiere ver la validez / invalidez de una forma argumental, hay que examinar todas las instancias de sustitución posibles, para ver si alguna tiene premisas verdaderas y conclusiones falsas. Para facilitar las cosas, las instancias se pueden representar en una tabla de verdad (ver las de arriba).

Las tablas de verdad son un método puramente mecánico para decidir validez o invalidez de cualquier argumento.

Podemos poner como ejemplos algunas formas de argumento que tienen enunciados condicionales:

Modus Ponens

p	q	p → q
T	T	T
T	F	F
F	T	T
F	F	T

Modus Tollens

p	q	p ⊃ q	~q	~p
T	T	T	F	F
T	F	F	T	F
F	T	T	F	T
F	F	T	T	T

Silogismo Hipotético

p	q	r	p ⊃ q	q ⊃ r	p ⊃ r
T	T	T	T	T	T
T	T	F	T	F	F
T	F	T	F	T	T
T	F	F	F	T	F
F	T	T	T	T	T
F	T	F	T	F	T
F	F	T	T	T	T
F	F	F	T	T	T

Tambien puede servir saber que existen dos formas de argumento parecidas al Modus Ponens y Tollens, pero son inválidas, las falacias de afirmación del consecuente y negación del antecedente.

p ⊃ q		p ⊃ q
q	y	~p
∴ p		∴ ~q

Comprobándolo en una misma tabla de verdad:

p	q	p ⊃ q	~p	~q
T	T	T	F	F
T	F	F	F	T
F	T	T	T	F
F	F	T	T	T

Formas sentenciales

Una forma sentencial es cualquier sucesión de símbolos que contiene variables sentenciales, de manera que, al sustituir enunciados por las variables sentenciales, se obtiene un enunciado.

Cabe recalcar que por convención, la primera forma sentencial es p, luego q, y así sucesivamente.

Cuando una forma sentencial solo tiene instancias de sustitución verdaderas se conoce como una tautología.

Por ejemplo, `p v ~p` :

p	~p	p v ~p
T	F	T
F	T	T

Tambien existen enunciados formalmente falsos, como `p . ~p`. Un enunciado se contradice con otro enunciado cuando es lógicamente imposible que ambos sean verdaderos. Cuando una forma sentencial solo tiene instancias de sustitución falsas, es una contradicción.

Los enunciados y formas sentenciales que no son tautologías ni contradicciones, son contingencias.

Cuando dos enunciados tienen el mismo valor de verdad, son materialmente equivalentes, y se representa con el símbolo de tres barras horizontales (≡). Ejemplo:

p	q	p ≡ q
T	T	T
T	F	F
F	T	F
F	F	T

Unas equivalencias lógicas que son importantes son los Teoremas de Morgan:

```
~(p . q) ≡ (~p v ~q)

~(p v q) ≡ (~p . ~q)
```

La tabla de verdad del primero es la siguiente:

p	q	p.q	~(p.q)	~p	~q	~p v ~q	~(p.q) ≡ (~p v ~q)
T	T	T	F	F	F	F	T
T	F	F	T	F	T	T	T
F	T	F	T	T	F	T	T
F	F	F	T	T	T	T	T

Método de deducción

Prueba formal de validez

A medida que aumenta la cantidad de enunciados, se hace cada vez más largo probar un argumento por la tabla de verdad (2^n).

Una prueba formal de validez para un argumento se define como una sucesión de enunciados, cada uno siendo una premisa de ese argumento o bien se sigue de los precedentes por un argumento válido elemental, y tal que el último enunciado de la secuencia es la conclusión del argumento.

El argumento válido elemental es cualquier argumento que es una instancia de sustitución de una forma de argumento válida. Más adelante se van a detallar algunas formas de argumento válidas elementales (reglas de inferencia).

Por ejemplo:

$$\sim C \supset (D \supset E)$$
$$\sim C$$
$$\therefore (D \supset E)$$

Es un argumento válido elemental, ya que es una instancia de sustitución de Modus Ponens:

$$p \supset q$$
$$p$$
$$\therefore q$$

Reglas de inferencia: Estas son formas válidas elementales de argumentos, se pueden usar para construir pruebas formales de validez para argumentos más complicados.

```
1.  1. Modus Ponens (M.P.)
2.
3.      p ⊃ q
4.      p
5.      ∴ q
6.
7.  2. Modus Tollens (M.T.)
8.
9.      p ⊃ q
10.     ~q
11.     ∴ ~p
12.
13. 3. Silogismo Hipotético (H.S.)
14.
15.     p ⊃ q
16.     q ⊃ r
17.     ∴ p ⊃ r
18.
19. 4. Silogismo Disyuntivo (D.S.)
20.
21.     p v q
22.     ~p
23.     ∴ q
24.
25. 5. Dilema Constructivo (C.D.)
26.
27.     (p ⊃ q) . (r ⊃ s)
28.     p v r
29.     ∴ q v s
30.
31. 6. Dilema Destructivo (D.D.)
32.
```

```
    (p ⊃ q) . (r ⊃ s)
    ~q v ~s
    ∴ ~p v ~r

7. Simplificación (Simp.)

    p . q
    ∴ p

8. Conjunción (Conj.)

    p
    q
    ∴ p . q

9. Adición (Ad.)

    p
    ∴ p v q
```

Regla de reemplazo

Consiste en reemplazar una parte cualquiera de un enunciado compuesto por una expresión que es lógicamente equivalente. La podemos adoptar como un principio adicional de inferencia.

```
10. Teoremas de De Morgan (De M.):

    ~(p . q) ≡ (~p v ~q)
    ~(p v q) ≡ (~p . ~q)

11. Conmutación (Conm.):

    (p v q) ≡ (q v p)
    (p . q) ≡ (q . p)

12. Asociación (Asoc.):

    [p v (q v r)] ≡ [(p v q) v r]
    [p . (q . r)] ≡ [(p . q) . r]

13. Distribución (Dist.):

    [p . (q v r)] ≡ [(p . q) v (p . r)]
    [p v (q . r)] ≡ [(p v q) . (p v r)]

14. Doble Negación (D.N.):

    p ≡ ~~p
```

```
15. Transposición (Trans.):

    (p ⊃ q) ≡ (~q ⊃ ~p)

16. Implicación Material (Impl.):

    (p ⊃ q) ≡ (~p v q)

17. Equivalencia Material (Equiv.):

    (p ≡ q) ≡ [(p ⊃ q) . (q ⊃ p)]
    (p ≡ q) ≡ [(p . q) v (~p . ~q)]

18. Exportación (Exp.):

    [(p . q) ⊃ r] ≡ [(p ⊃ (q ⊃ r)]

19. Tautología (Taut.):

    p ≡ (p v p)
    p ≡ (p . p)
```

Hay algunas diferencias importantes entre las primeras nueve y estas reglas de inferencia:

- Las primeras nueve se pueden aplicar a renglones enteros de una demostración.
- Las otras se pueden aplicar a renglones enteros o a partes de renglones.

Demostración de la invalidez

Estas 19 reglas son incompletas, es decir, hay argumentos válidos función de verdad que no se pueden demostrar usando tan solo estas reglas.

Regla de demostración condicional

Se puede aplicar a argumentos cuyas conclusiones sean enunciados condicionales. Una demostración con esta regla se construye suponiendo que el antecedente de su conclusión es una premisa adicional, y luego deduciendo el consecuente de su conclusión por una sucesión de argumentos válidos elementales. Como un ejemplo:

$$(A \vee B) \supset (C.D)$$
$$(D \vee E) \supset F$$
$$\therefore A \supset F$$

Se escribe:

1. (A v B) ⊃ (C.D)		
2. (D v E) ⊃ F	/ ∴	A ⊃ F

3. A	/ ∴	**F (C.P.)**
4. A v B		3, Ad.
5. C.D		1, 4, M.P.
6. D.C		5, Conm.
7. D		6, Simpl.
8. D v E		7, Ad.
9. F		2, 8, M.P.

Regla de demostración indirecta o por reducción al absurdo

Se supone como premisa adicional la negación de la conclusión, y se deduce una contradicción explicita. Si el supuesto lleva a una contradicción, el supuesto debe ser falso, y su negación debe ser verdadera. Ejemplo:

A ⊃ (B.C)

(B v D) ⊃ E

D v A

∴ E

Puede escribirse:

1. A ⊃ (B.C)		
2. (B v D) ⊃ E		
3. D v A		/ ∴ E
4. ~E		**IP (Dem. Indirecta)**
5. ~(B v D)		2, 4, M.T.
6. ~B. ~D		5, De M.
7. ~D. ~B		6, Conm.
8. ~D		7, Simpl.
9. A		3, 8, D.S.
10. B.C		1, 9, M.P.
11. B		10, Simpl.
12. ~B		6, Simpl.
13. B. ~B		11,12, Conj.

El último renglón es una contradicción explícita, por lo tanto, el argumento original es válido.

Lógica de predicados

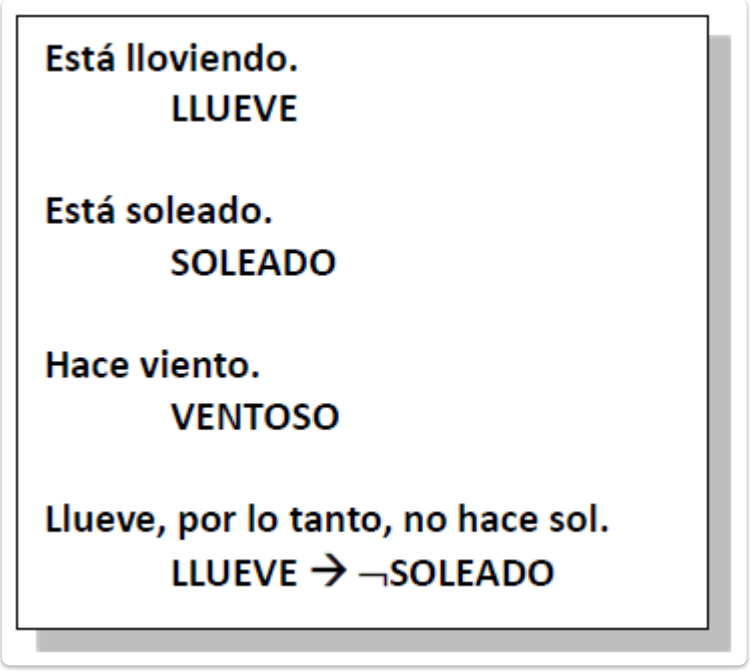
Introducción, concepto de semidecidible

La lógica de predicados es apropiada para representar el conocimiento, pero tambien está la cuestión de si ofrece un mecanismo adecuado para razonar con ese conocimiento. Esto en primera instancia parece cierto, ya que se pueden deducir nuevas sentencias a partir de las anteriores, pero no se tiene un procedimiento de decisión. Existen procedimientos que permitirán encontrar la prueba de un teorema dado si es que en realidad se trata de un teorema. Pero no está garantizado que estos procedimientos paren cuando la sentencia no sea un teorema.

Aunque la lógica de predicados de primer orden no es decidible, si es semidecidible. La lógica de predicados, a pesar de su indecidibilidad teórica, puede resultar útil como medio de representación y manipulación de ciertos tipos de conocimiento.

representación de hechos simples en lógica

Con lógica proposicional, es fácil representar hechos del mundo real como proposiciones lógicas. Ejemplo:



Como otro ejemplo: Si quiero representar la frase *Todos los hombres son mortales* en lógica proposicional, podría usar *HOMBREMORTAL*, pero esta forma no refleja exactamente el hecho de que todos y cada uno de los hombres son mortales. Para hacerlo, es necesaria la lógica de predicados, ya que permite representar cosas que no serían representables de forma razonable usando lógica proposicional.

Un ejemplo que usa lógica de predicados:

1. Marco era un hombre.
2. Marco era un pompeyano.
3. Todos los pompeyanos eran romanos.
4. César fue un gobernante.
5. Todos los romanos o eran leales a César o le odiaban.
6. Todo el mundo es leal a alguien.
7. La gente solo intenta asesinar a los gobernantes a los que no es leal.
8. Marco intentó asesinar a César.

Pasando las 5 primeras sentencias:

```
1. hombre(Marco)
2. pompeyano(Marco)
3. ∀x: pompeyano(x) -> romano(x)
4. gobernante(cesar)
5. ∀x: romano(x): leal(x, cesar) v odia(x, cesar)
```

Hay algunas cuestiones involucradas en la conversión de lenguaje natural a sentencias lógicas:

- Muchas frases son ambiguas.
- Suele haber más de una forma de representar el conocimiento.
- No es probable que en un conjunto de frases esté contenida toda la información necesaria para razonar sobre el tema en cuestión.

representación de relaciones instancia y es-un

En el ejemplo anterior estas relaciones estaban implícitas, si las hacemos explícitas:

```
1. instancia(Marco, hombre)
2. instancia(Marco, pompeyano)
3. es-un(pompeyano, romano)
4. instancia(César, gobernante)
5. ∀x: instancia(x, romano) -> leal(x, César) v odia(x, César)
6. ∀x: ∀y: ∀z: instancia(x, y) ^ es-un(y, z) -> instancia(x, z)
```

Entonces:

- No es necesario representar explícitamente estas relaciones, ya que muchas veces termina siendo engorroso.
- Como se ve, se pueden obtener varias representaciones de un determinado hecho en un sistema de representación dado.

representación de funciones calculables y predicados computables

Básicamente, es incorporar sumas, restas, igualdades, para poder responder a algunas cuestiones de una manera más simple.

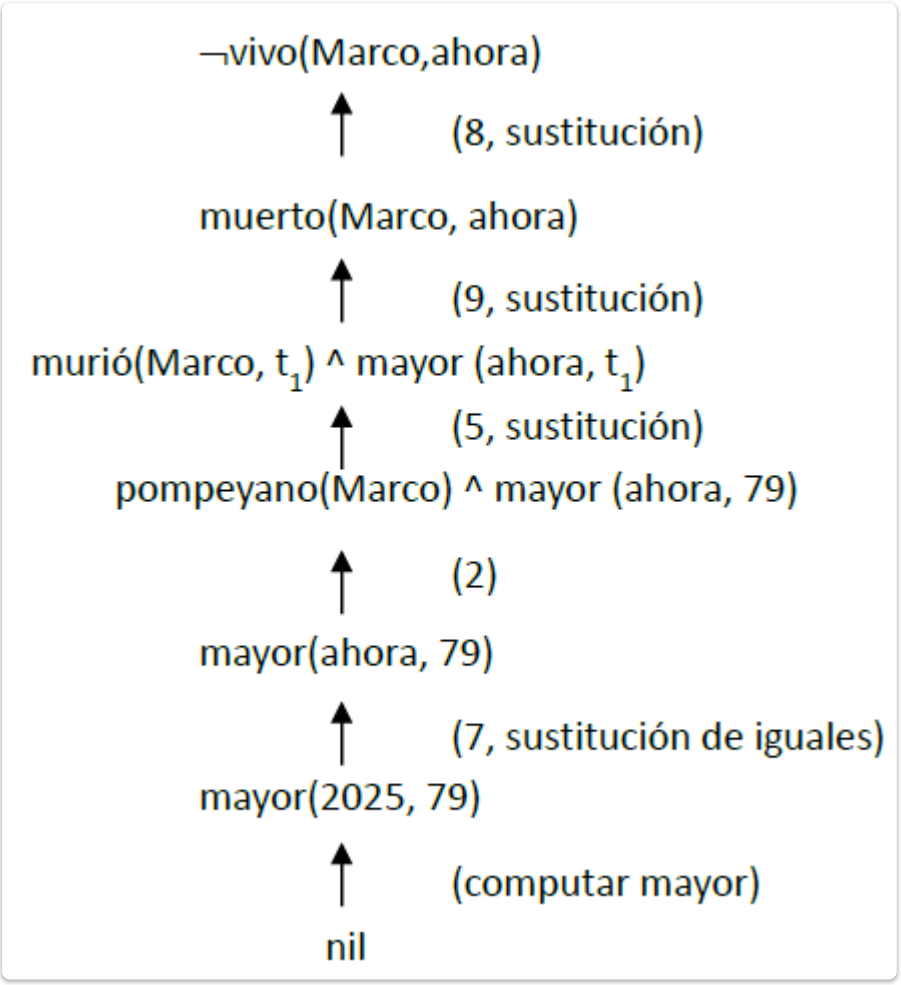
Ejemplo, se tiene este conjunto de hechos:

1. hombre(Marco)
2. pompeyano(Marco)
3. nace(marco,40)
4. ∀x: hombre(x) → mortal(x)
5. ∀x: [pompeyano(x) → murió(x,79)]
   erupción(volcán,79)
6. ∀x: ∀t1: ∀t2: mortal(x) ^ nace(x, t1) ^ mayor (t2 – t1,150) → muerto(x, t2)
7. ahora = 2025
8. **∀x: ∀t: [vivo(x,t) → ¬muerto(x,t)] ^ [¬muerto(x,t) → vivo(x,t)]**
9. **∀x: ∀t1: ∀t2: murió(x, t1) ^ mayor (t2 , t1) → muerto(x, t2)**

Si me preguntara si Marco está vivo, habría que demostrar:

¬vivo(Marco,ahora)

Demostrando:



Método de resolución

La resolución es un procedimiento que obtiene demostraciones por refutación: para probar una proposición, se intente demostrar que su negación lleva a una contradicción con las proposiciones conocidas.

Conversión a forma clausal

Forma normalizada conjuntiva: Hace que una fórmula tenga menos anidamiento entre expresiones, y separa los cuantificadores de la fórmula.

Para que la resolución funcione, hay que transformar el conjunto de fórmulas bien formadas (fbf) a un conjunto de cláusulas, donde una cláusula es una fbf en forma normalizada conjuntiva que no tiene conectivas ^.

Hay un algoritmo de conversión a forma clausal que voy a obviar porque no veo que valga la pena resumirlo acá. Ver el apunte.

Las bases de la resolución

Se comparan (resuelven) dos cláusulas llamadas padres, produciendo una inferida de ellas, que representa la forma en que las dos cláusulas padres interactúan entre sí.

Supongamos:

```
invierno v verano
¬invierno v frío
```

Siempre será cierta una de las aserciones `invierno` o `~invierno`. Entonces a partir de estas dos se puede deducir `verano v frío`.

El procedimiento de resolución hace lo siguiente:

- Toma dos cláusulas que tienen el mismo literal, una en forma positiva y la otra en negativa.
- El resolvente se obtiene combinando todos los literales de las dos cláusulas padres excepto aquellos que se cancelan.
- Si la cláusula producida es la cláusula vacía, es que se ha encontrado una contradicción.

Resolución en lógica proposicional

Hay un algoritmo de resolución de proposiciones:

1. Convertir todas las proposiciones de F a forma clausal.
2. Negar P y convertir el resultado a forma clausal. Añadir la cláusula resultante al. conjunto de cláusulas obtenidas en el paso 1.
3. Hasta que se encuentre una contradicción o no se pueda seguir avanzando repetir:.
   a. Seleccionar dos cláusulas. Llamarlas cláusulas padres.
   b. Resolverlas juntas. La cláusula resultante, llamada resolvente, será la disyunción de todos los literales de las cláusulas padres con la siguiente excepción: si existen pares de literales L y ¬L, de forma que una de las cláusulas padre contenga L y la otra contenga ¬L, entonces se eliminarán tanto L como ¬L del resolvente.
   c. Si el resolvente es la cláusula vacía, es que se ha encontrado una contradicción. Si no lo es, añadirla al conjunto de cláusulas disponibles.

Algoritmo de unificación

Para encontrar contradicciones, se necesita un procedimiento que compare dos literales y descubra si existe algún conjunto de sustituciones que los haga idénticos. Este procedimiento es el algoritmo de unificación.

- Para unificar dos literales, primero se comprueba si los predicados coinciden.
- Luego, se van comprobando los argumentos de dos en dos. Para comprobar cada par de argumentos, basta con llamar recursivamente al procedimiento de unificación. Aquellos predicados o constantes que sean diferentes no se pueden emparejar; aquellos que sean idénticos pueden hacerlo. Una variable se puede emparejar con otra variable, con cualquier constante o con un predicado, con la restricción de que el predicado no debe contener ninguna instancia de la variable con la que se está emparejando.

El objetivo del procedimiento de unificación es descubrir, al menos una sustitución que permita el emparejamiento de dos literales

Resolución en lógica de predicados

Usando el siguiente algoritmo:

1. Convertir todas las sentencias de F a forma clausal.
2. Negar P y convertir el resultado a forma clausal. Añadirlo al conjunto de cláusulas obtenidas en 1.
3. Hasta que se encuentre una contradicción o no pueda realizarse ningún progreso o se haya realizado una cantidad de esfuerzo predeterminada, repetir: .
    a. Seleccionar dos cláusulas. Llamarlas cláusulas padres.
    b. Resolverlas. El resolvente será la disyunción de todos los literales de ambas cláusulas padres, una vez realizadas las sustituciones apropiadas, con la siguiente excepción: si existe un par de literales T1 y ¬T2 tales que una de las cláusulas padres contenga T1 y la otra contenga T2 y si T1 y T2 son unificables, entonces ni T1 ni T2 deben aparecer en el resolvente. Llamaremos a T1 y T2 literales complementarios. A continuación, utilizar la sustitución producida por la unificación para crear el resolvente. Si existe más de una pareja de literales complementarios, en el resolvente solo se eliminará uno de ellos.
    c. Si el resolvente es la cláusula vacía, se ha encontrado una contradicción. Si no lo es, añadirla al conjunto de cláusulas sobre las que se está aplicando el procedimiento.

Cuando la elección de cláusulas se hace siguiendo ciertas formas sistemáticas, el proceso encontrara una contradicción si es que existe.

representación del conocimiento mediante reglas

Comparando conocimiento procedimental y conocimiento declarativo

Una representación declarativa es aquella en donde el conocimiento está especificado, pero no viene acompañado de una manera en que dicho conocimiento debe ser usado. Para usarlas, se deben aumentar con un programa que especifique lo que puede hacerse con el conocimiento, y de qué modo.

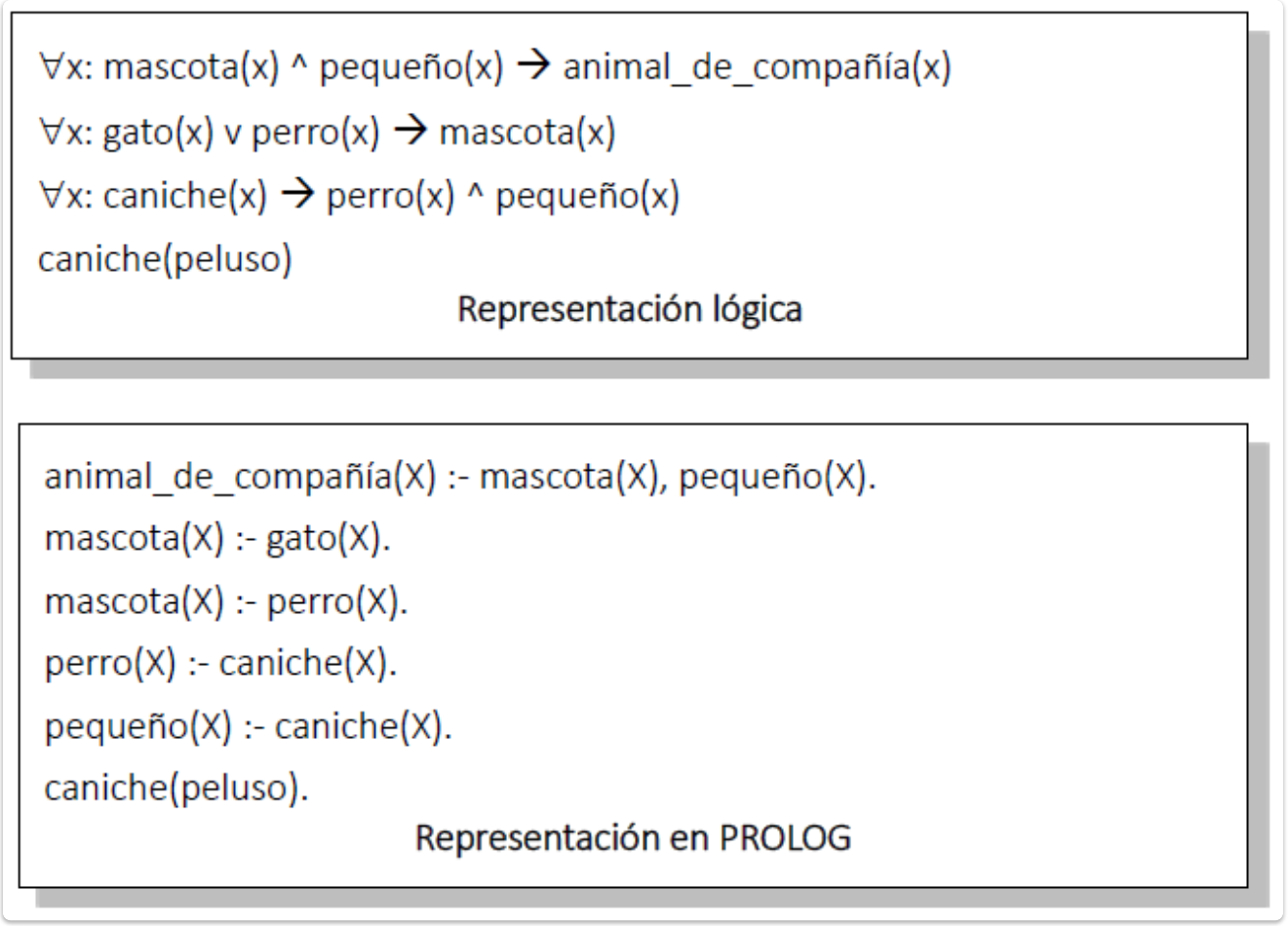
Una representación procedimental es aquella en la que la información de control necesaria para usar el conocimiento se encuentra embebida en el propio conocimiento. Para usarla se necesita aumentarla con un intérprete que siga las instrucciones dadas por el conocimiento.

La diferencia entre el punto de vista declarativo y el procedimental radica en donde se encuentra la información de control.

Programación lógica

Es un paradigma en donde las aserciones lógicas se consideran como programas. El sistema más popular es PROLOG. Un programa PROLOG se describe como una serie de aserciones lógicas, cada una de las cuales es una cláusula de Horn. Una cláusula de Horn tiene como mucho un literal positivo. Esto tiene dos consecuencias:

1. Debido a la representación uniforme, se puede escribir un sencillo y potente interprete que resulte eficaz.
2. La lógica de los sistemas de cláusulas de Horn es decidible.



Tenemos hechos y reglas:

- Los hechos representan sentencias acerca de objetos específicos, solo contienen constantes.
- Las reglas representan sentencias acerca de diferentes clases de objetos, contienen variables.

Una diferencia importante es que PROLOG fija una estrategia de control (textual):

La estrategia de control PROLOG básica, comienza con una sentencia problema, que en este caso es considerado como el objetivo a probar; busca las aserciones que pueden probar el objetivo; considera hechos que prueban el objetivo directamente, y además considera cualquier regla cuya cabeza se empareje con el objetivo. Para decidir cuándo puede aplicar una regla o un hecho al problema que le ocupa utiliza el procedimiento de unificación estándar. Razonará hacia atrás desde ese objetivo, hasta que encuentre el modo de terminar con las aserciones en el programa. Considera los caminos utilizando la estrategia de la búsqueda primero en profundidad, así como utilizando la vuelta atrás. En cada punto en el que debe elegir, considera las opciones siguiendo el orden en que estas aparecen en el programa. Si el objetivo tiene más de una parte de conjuntiva, comprueba las partes en el orden en que estas aparecen, propagando los enlaces de las variables que determinó la unificación.

Razonamiento hacia delante y hacia atrás

Si bien PROLOG efectúa la búsqueda a partir de un estado objetivo, en realidad existen dos direcciones hacia las que se puede dirigir, hacia delante a partir de los estados iniciales, o hacia atrás partiendo de los estados objetivos.

Hacia delante

Se construye un árbol de secuencias de movimientos que se pueden presentar como soluciones, empezando por la configuración inicial en la raíz. Se genera el siguiente nivel encontrando todas las reglas cuyos lados izquierdos se relacionen con el nodo raíz, y que usen sus lados derechos para crear nuevas configuraciones. Se continúa hasta que se consiga una configuración que se empareje con el estado objetivo.

Hacia atrás

Se construye un árbol de secuencias de movimientos que ofrezcan soluciones empezando con la configuración objetivo en la raíz del árbol. Se genera el siguiente nivel encontrando las reglas cuyos lados derechos están ligadas con el nodo raíz. Se usa el lado izquierdo para generar los nodos. Se continúa hasta que se genera un nodo que se empareja con el estado inicial.

Razonamiento simbólico bajo incertidumbre

Razonamiento no monótono

Las técnicas de razonamiento anteriores sirven para modelos del mundo completo, consistente e inalterable, sin embargo, muchas veces no es posible crear estos modelos. Para manejar este problema, hay dos enfoques:

- Razonamiento no monótono: Los axiomas o las reglas de inferencia se extienden para que sea posible razonar con información incompleta.
- Razonamiento estadístico: Se extiende la representación para permitir algún tipo de medida numérica sobre la certeza para asociar a cada sentencia.

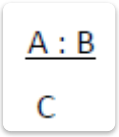
Los sistemas convencionales de razonamiento están hechos para trabajar con información que cumple tres propiedades:

1. Es completa con respecto al dominio de interés.
2. Es consistente.
3. La única forma de cambiar la información es que se añadan nuevos hechos.

Si no se verifican las tres, estos sistemas resultan inadecuados. Para eso están los sistemas de razonamiento no monótono.

Razonamiento por defecto: Pretende llegar a conclusiones basadas en lo que es más probable que sea cierto. Tenemos dos enfoques:

1. Lógica no monótona: Se proporciona una base para razonar por omisión, mediante un operador modal `M`, que significa "es consistente".
2. Lógica por defecto: Introduce un nuevo tipo de reglas de inferencia (ver imagen). Esto se lee como "si A es probable y es consistente asumir B, entonces se concluye que C".



Razonamiento minimalista: Parte de la idea de que hay que asumir como verdaderas únicamente las cosas necesarias para mantener la base de conocimiento consistente. Si algo no está respaldado por la información disponible, es falso o no se asume verdadero.

La suposición de mundo cerrado aplica esta idea a bases de datos: si un dato no aparece, se considera que no es cierto. Pero esto puede fallar cuando la base de datos está incompleta. La circunscripción busca mejorar este razonamiento, considerando las relaciones entre distintos predicados y limitando las situaciones en las que estos pueden ser verdaderos.

Razonamiento estadístico

El razonamiento estadístico permite trabajar con situaciones en las que no se puede determinar con certeza si algo es verdadero o falso, pero se puede estimar que tan probable es. Tiene utilidad en problemas donde existe el azar o situaciones donde pueden existir excepciones.

Las estadísticas bayesianas permiten actualizar la probabilidad de una hipótesis cuando aparece nueva evidencia. El Teorema de Bayes relaciona la probabilidad previa de una hipótesis con la probabilidad de observar determinada evidencia si esa hipótesis fuera cierta.

El principal problema es que, a medida que aumenta la cantidad de variables, la cantidad de probabilidades necesarias crece rápidamente. Esto hace difícil obtener, almacenar y calcular todas las probabilidades. A pesar de estas limitaciones, el razonamiento bayesiano es una base importante para trabajar con incertidumbre, y existen métodos que lo hacen más práctico.

Factores de certeza

Los factores de certeza son una forma simplificada de trabajar con incertidumbre sin tener que utilizar un sistema bayesiano completo. En MYCIN, el conocimiento se representa mediante reglas a las que se les asigna un valor que indica qué tan fuerte es la evidencia a favor o en contra de una conclusión. Estos valores pueden combinarse cuando existen varias evidencias o reglas relacionadas, permitiendo obtener una estimación de confianza sobre una conclusión.

Redes bayesianas

Las redes bayesianas permiten aplicar el razonamiento probabilístico sin tener que calcular las probabilidades de todas las combinaciones posibles. Para ello, aprovechan que muchos eventos son condicionalmente independientes y representan únicamente las relaciones relevantes entre grupos de eventos, reduciendo considerablemente la complejidad del modelo.

Teoría de Dempster-Shafer

La teoría de Dempster-Shafer representa la incertidumbre mediante un intervalo [creencia, verosimilitud] en lugar de asignar una única probabilidad. La creencia indica cuánto apoya la evidencia a una hipótesis, mientras que la verosimilitud indica cuánto espacio queda para que esa hipótesis sea cierta. A medida que se obtiene más evidencia, el intervalo se reduce, reflejando que existe mayor información y confianza.

Sistemas semánticos para la representación del conocimiento

Una representación del conocimiento es una forma de organizar la información para que un problema pueda resolverse más fácilmente. Una buena representación elimina detalles innecesarios y hace visibles las relaciones importantes.

Estructuras de ranura y relleno débiles

Son estructuras que organizan el conocimiento en entidades y sus atributos, facilitando especialmente la herencia de propiedades. Las principales formas son las redes semánticas y los marcos (frames).

Redes semánticas

Las redes semánticas representan el conocimiento mediante nodos conectados por enlaces etiquetados, donde los nodos representan objetos o conceptos y los enlaces representan relaciones entre ellos. Permiten recorrer las conexiones para encontrar relaciones y representar información similar a la lógica de predicados.

Marcos

Un marco (frame) representa una entidad mediante un conjunto de atributos o ranuras (slots) que tienen valores asociados. Los marcos pueden conectarse entre sí y permiten representar clases e instancias, facilitando la herencia de propiedades desde una clase hacia sus elementos.

Estructuras de ranura y relleno fuertes

Son estructuras que, además de organizar el conocimiento, incorporan reglas más específicas sobre cómo representarlo y utilizarlo. Entre ellas se encuentran la Dependencia Conceptual, los Guiones y CYC.

Dependencia conceptual

La Dependencia Conceptual (DC) busca representar el significado de eventos expresados mediante lenguaje natural de una manera independiente del idioma utilizado. Para ello utiliza un conjunto específico de primitivas conceptuales, que permiten representar información y facilitar la obtención de inferencias.

Guiones

Los guiones (scripts) representan secuencias habituales de eventos dentro de un contexto determinado. Describen cómo normalmente se desarrollan las situaciones, conectando los eventos mediante relaciones causales: unos eventos permiten o provocan que ocurran los siguientes.

CYC

CYC fue un proyecto orientado a construir una gran base de conocimiento que representara el sentido común humano, incluyendo información que normalmente se da por obvia y no se expresa explícitamente. Su objetivo era combinar este conocimiento general con bases especializadas para crear sistemas de IA más completos y resistentes. A diferencia de la Dependencia Conceptual, CYC no se limita a representar eventos, sino también objetos, actitudes y otros tipos de conocimiento, teniendo además en cuenta cómo manejar bases de conocimiento con millones de elementos.

.
.
.
.
.
.
.
.
.
.
.
.
.
.
.
.
.
.
.
.
.

Anexo: Las 19 reglas de inferencia

1. Modus Ponens (M.P.)

   p ⊃ q
   p
   ∴ q
2. Modus Tollens (M.T.)

   p ⊃ q
   ~q
   ∴ ~p
3. Silogismo Hipotético (H.S.)

   p ⊃ q
   q ⊃ r
   ∴ p ⊃ r
4. Silogismo Disyuntivo (D.S.)

   p v q
   ~p
   ∴ q
5. Dilema Constructivo (C.D.)

   (p ⊃ q) . (r ⊃ s)
   p v r
   ∴ q v s
6. Dilema Destructivo (D.D.)

   (p ⊃ q) . (r ⊃ s)
   ~q v ~s
   ∴ ~p v ~r
7. Simplificación (Simp.)

   p . q
   ∴ p
8. Conjunción (Conj.)

   p
   q
   ∴ p . q
9. Adición (Ad.)

   p
   ∴ p v q

A partir de aca son las reglas de reemplazo

10. Teoremas de De Morgan (De M.):

    ~(p . q) ≡ (~p v ~q)
    ~(p v q) ≡ (~p . ~q)
11. Conmutación (Conm.):

    (p v q) ≡ (q v p)
    (p . q) ≡ (q . p)
12. Asociación (Asoc.):

    [p v (q v r)] ≡ [(p v q) v r]
    [p . (q . r)] ≡ [(p . q) . r]
13. Distribución (Dist.):

    [p . (q v r)] ≡ [(p . q) v (p . r)]
    [p v (q . r)] ≡ [(p v q) . (p v r)]
14. Doble Negación (D.N.):

    p ≡ ~~p
15. Transposición (Trans.):

    (p ⊃ q) ≡ (~q ⊃ ~p)
16. Implicación Material (Impl.):

    (p ⊃ q) ≡ (~p v q)
17. Equivalencia Material (Equiv.):

    (p ≡ q) ≡ [(p ⊃ q) . (q ⊃ p)]
    (p ≡ q) ≡ [(p . q) v (~p . ~q)]
18. Exportación (Exp.):

    [(p . q) ⊃ r] ≡ [(p ⊃ (q ⊃ r)]
19. Tautología (Taut.):

    p ≡ (p v p)
    p ≡ (p . p)